

This is a changelog regarding the reasons for the change in the final prototype.

1. User Interface & User Experience UI/UX: In my original design document, I had made sure that I made use of essential white spacing, clear branding and common conventions. However, while I was done developing the first prototype, the design was more cluttered than expected. The accent colors are everywhere, and the layout was not intuitive, with lots of inconsistencies as the brand colors were everywhere. I also noticed how the color contrasts were very poor as the earlier shade of color scheme I had chosen were very poor. With that considered, I had made sure to reduce the use of Brand Coloring when not applicable, and limited the styling to only call to action elements like
 - Book Meeting Buttons
 - Logo Branding
 - Highlighting specific words to draw user attentionAfter these iterative changes were made I decided to follow through with it with a more minimalist and consistent layout. Taking into account color contrasting, hierarchy and use of whitespace, resulting in a better prototype.

To be more precise, the original color pallet for the brand, <https://www.htmlcsscolor.com/hex/E3FAE3>. Combining this with the background layout, which is also white, made the text within it very hard to read, which goes against the WCAGG Guideline principle. Changing the color to #6CB26C, rgb(108,178,108) made it more readable and clearer, passing the contrast testing in DevTools in Chrome. The website looked more vibrant and paired greatly with the company's ethos of Green Energy Technology.

Also, I HAD ensured replaced the images to make sure they suit the company's services more. All the images were gotten from open-source websites as stated in my Asset Log Documentation; majority of these sites were:

- Cosmos for their bold and clear pictures. They offer tons of free images for different users as they also partner with Pinterest
- LummiAi is another website I got my assets from. They provide stock images generated by AI so all images are Copyright Free
- The Icon I used was from Bootstrap Icons another open-source platform

For a company that needs to clearly pass their ethos to their customer quickly and easily these sources provided images that were very self-descriptive.

Other UI Elements I changed include form fields, buttons, links, color branding and new interactive features for example:

- Buttons now have an active state that users will be able to distinguish when they are interacting with it or not
- Links now have a generic style applied when a user interacts with it that makes sure that the link gets highlighted
- Form Fields now have more styling that makes sure they are easily scannable and navigable making it easy to understand its requirements
- The font style from "Montserrat" to "Poppins" was necessary as the Montserrat font portrayed the website as less of a business. Switching to another Sans-Serif Font, Poppins which is also an open-source font provided a more simple and clean aesthetic, while also promoting the Brand and website as a business.

Other features that were not included in the Original Proposal include the following:

- Third Party Libraries :
 1. FlatPickr: This is an open-source software that allows for dynamic rendering of dates for a web services. It's compatible with different tech stacks including tech stacks without frameworks. It has been stable since it's last release and does not have vulnerabilities
 2. ChartJS, which is also another open-source software that renders dynamic graphs and data without having to implement the barebones, as it comes with styling, functionality and other complex features like tool tips, leaving room for customizations like fonts, stylings and type of charts to be shown. It has several releases over the couple of years, and I'm using their most recent one as at 2025. However, to avoid over bloating the tech stack and increasing dependency on third party libraries, I decided to use this via the CDN, Content Delivery Network, making sure that performance is not sacrificed. If the CDN was to go down, the browsers can default back to their original styling
 3. Jest: Jest is an open-source framework that allows for automated testing for API's. I used it to test the server I built and ensure proper error messages are delivered as appropriate. I also tested the VSCode server to make sure that it provides the index page upon start up.

Another major change was in the database schema. The previous schema was well laid out with standardized normalization parts but considering how the database will scale and how much redundant data will be stored; I decided to change it. For example, if a user who has no bookings, would only want to check their carbon footprint, learn more about the

company or interested in the Newsletter service. Taking anything apart from their email address for this purpose will lead to redundant and excess amount of data in the database. To address this issue, I decided that only people who make bookings can make accounts, this way, we know that we only have two kinds of users in the database, BOOKINGS and NEWSLETTER customers.

I also intended to make sure that the system can confirm a user by sending them a temporary password to the email they provided, so that upon login they can update their password as taking their booking details, and asking for account creation at the same time is bad User Experience. This method makes sure that the user with the registered email is valid.

But since I was not able to fully implement that, I required passwords from the users directly, so they could just login with the same password used. I had also made another iteration to the database schema as I realized that, if a user wanted to make more bookings, it would simply overwrite the previous one and cause more data integrity issues and problems.

So I decided to make a new table that stored new bookings for users based on their userId, to avoid redundancy. I also made the bookings column store data as JSON type; here is a representation of such schema.

```
```
```

```
create table newBookings (userId foreign_key not null, id primary key,
auto_increment, booking JSON);
```

```
```
```

This further store new bookings in an array -like structure as how a NoSQL database would store it, and would demo something like this: